

HASHDIT

HashDit

Smart Contract Security

Venus SpokeHub

Security Audit Report

Venus

7 Sep 2026 – 11 Sep 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	7 Sep 2026 - 11 Sep 2026
Overall Risk	Low
Project Name	Venus SpokeHub Audit
Github	https://github.com/VenusProtocol/isolated-pools/pull/559 https://github.com/VenusProtocol/protocol-reserve/pull/168
Commit	PR559: 7e3c344d5dcc60e5829ff583f6a1e2218112cb6b PR168: a7b8f12de27657378d4173432852c70b000339e2

Smart Contract List

No.	Contract Name	Verdict	Details
1	contracts/Spoke/SpokeComptroller.sol	Low	[L01]-Fixed [I01]-Ack
2	contracts/Spoke/SpokeComptrollerInterface.sol	Informational	[I03]-Fixed
3	contracts/Spoke/SpokeComptrollerStorage.sol	-	-
4	contracts/Lens/SpokePoolLens.sol	-	-
5	contracts/Shortfall/Shortfall.sol	Low	[L01]-Fixed
6	deploy/024-deploy-spoke-pool-registry.ts	Low	[L01]-Fixed [I02]-Fixed
7	deploy/025-deploy-spoke-comptroller.ts	Informational	[I02]-Fixed
8	deploy/026-deploy-spoke-pool-lens.ts	-	-
9	deploy/027-deploy-spoke-vtoken-beacon.ts	-	-
10	deploy/028-deploy-spoke-vtokens.ts	Low	[L01]-Fixed [I04]-Ack
11	helpers/spokeDeploymentConfig.ts	Informational	[I04]-Ack
12	contracts/ProtocolReserve/ProtocolShareReserve.sol	Low	[L01]-Fixed

Findings

[L01] Registry separation severs the protocol income path, allowing the pool to launch in a state where no liquidation can complete	
Contract	SpokeComptroller.sol, contracts/Shortfall/Shortfall.sol, deploy/024-deploy-spoke-pool-registry.ts, deploy/028-deploy-spoke-vtokens.ts ProtocolShareReserve.sol (protocol-reserve)
Severity	Low

Description	This PR deploys a dedicated SpokePoolRegistry for the spoke pool (the same source as the isolated-pools PoolRegistry, but a separate instance with its own storage) and binds it into SpokeComptroller as an immutable. supportMarket accepts calls only from that registry:
	
	JavaScript
	`// SpokeComptroller – constructor and supportMarket`
	`address public immutable poolRegistry;`
	`function supportMarket(VToken vToken) external {`
	`_checkSenderIs(poolRegistry);`
	`...`
	`newMarket.isListed = true;`
	`}`
	
	PoolRegistry.addMarket is the only writer of the _vTokens mapping, and it must call comptroller.supportMarket(vToken) on the way. This pool's markets can therefore only ever be recorded in SpokePoolRegistry; the existing isolated-pools PoolRegistry can never hold them. That is a structural constraint imposed by the immutable, not a configuration choice.
	
	The downstream protocol-income components, however, each store a **single** registry address, and their setters replace rather than add:
	
	JavaScript
	`// ProtocolShareReserve – field, setter and membership check`
	`address public poolRegistry;`
	`function setPoolRegistry(address _poolRegistry) external onlyOwner {`
	`poolRegistry = _poolRegistry; // replaces; cannot hold two directories`
	`}`
	`// updateAssetsState`
	`if (`
	`comptroller != CORE_POOL_COMPTROLLER &&`

```
      IPoolRegistry(poolRegistry).getVTokenForAsset(comptroller, asset)
== address(0)
  ) revert InvalidAddress();
```

That check sits in VToken._seize after the collateral has already moved, and the call is unconditional:

JavaScript

```
  // VToken — _seize (unmodified by this PR)
  // EFFECTS & INTERACTIONS
  // (No safe failures beyond this point)
  accountTokens[borrower] = accountTokens[borrower] - seizeTokens;
  accountTokens[liquidator] = accountTokens[liquidator] +
liquidatorSeizeTokens;
  _doTransferOut(protocolShareReserve, protocolSeizeAmount);
  IProtocolShareReserve(protocolShareReserve).updateAssetsState(
      address(comptroller), underlying,
IProtocolShareReserve.IncomeType.LIQUIDATION
  );
```

The same single-registry pattern governs two further consumers, one of which — Shortfall — lives inside this repository:

JavaScript

```
  // Shortfall — _startAuction
  PoolRegistryInterface.VenusPool memory pool =
PoolRegistry(poolRegistry).getPoolByComptroller(comptroller);
  require(pool.comptroller == comptroller, "comptroller doesn't exist
pool registry");
  // RiskFundConverter — updateAssetsState
  if (!ensureAssetListed(comptroller, asset)) revert
MarketNotExistInPool(comptroller, asset);
```

getPoolByComptroller returns a zero-filled struct for an unknown comptroller, so that require necessarily fails.

Note that the team has already identified this issue. The closing comment in 024 states that *"ProtocolShareReserve support for more than one registry … has to be live before this registry is wired into it"*, and the repository contains a dedicated bscmainnet fork suite, tests/hardhat/Fork/HubSpoke/psrRegistryConflict.ts, that pins the conflict with assertions in both directions; its comments include *"Every liquidation in this pool would revert here, after the collateral has already moved"* and the explicit position *"Neither is a defect in this repo"*. This entry is recorded because the prerequisite is introduced

	by this PR's design decision — it did not exist beforehand — and **no code in this PR's changed files can prevent the pool from launching while it is unmet**, so it warrants explicit tracking and verification as a launch blocker. This is rated Low rather than higher because it is a launch-readiness precondition already identified by the team and pinned by an existing fork test suite, not an exploitable defect introduced by this PR's own code — the risk lies in launching before the prerequisite is met, not in the code itself. **Impact:** If the pool launches before the prerequisite is resolved, it loses every protocol-income path and all liquidation capability. All three seizure paths (liquidateBorrow, liquidateAccount, healAccount) route through _seize, so every liquidation reverts after the collateral has moved; Shortfall cannot start a bad-debt auction, removing the backstop at the same time; and RiskFundConverter rejects the pool's assets, so income cannot enter conversion and distribution. Bad debt cannot be cleared and the loss ultimately lands on the funder, the Liquidity Hub. The conflict is bidirectional: pointing the existing PSR at SpokePoolRegistry would break the existing isolated pools in exactly the same way.

Recommendation	Satisfy one of the following before launch, and assert that the pool resolves through every contract on the income path, not just the first one:
	1. Wait for multi-registry support to be live in ProtocolShareReserve, RiskFundConverter and Shortfall, then have governance register SpokePoolRegistry with each of them.
	2. Deploy a parallel ProtocolShareReserve / RiskFund / RiskFundConverter / Shortfall stack pointed at SpokePoolRegistry, and bind this pool's markets to it through VToken's per-market setters:
	
	JavaScript
	`// VToken - per-market binding (onlyOwner)`
	`vToken.setProtocolShareReserve(spokeProtocolShareReserve);`
	`vToken.setShortfallContract(spokeShortfall);`
	
	Option 2 is an alternative this audit proposes and is not covered in the team's own notes. If taken, RiskFundConverter must not be omitted: without it, assets reaching the ProtocolShareReserve cannot be converted and distributed.
	
	Whichever path is taken, add an assertion to the market-listing VIP that each vToken's configured protocolShareReserve and shortfallContract resolve this comptroller, rather than relying on the deployment scripts, which wire the chain's existing contracts by default.

Status	Fixed in protocol-reserve #168 (reviewed at a7b8f12, merged as 6f41e02). The PR keeps poolRegistry as the primary directory and adds a set of additional registries consulted after it, so once governance calls addPoolRegistry(SpokePoolRegistry) this pool resolves and its income can be reported. The storage layout was reviewed and is

upgrade-safe: all seventeen existing entries keep their slot, offset, label and type, and the two new variables are appended.

The fix does not, however, reach the whole income path. #168 changes ProtocolShareReserve alone, and that contract is only where income is booked. Where income is distributed, _releaseFund transfers to each configured destination and then calls back into it with the same comptroller, and RiskFundConverter re-checks that comptroller against its own single registry, which #168 leaves untouched:

```javascript
// RiskFundConverter._updateAssetsState - reached when the PSR releases funds
if (!ensureAssetListed(comptroller, asset)) revert MarketNotExistInPool(comptroller, asset);
```

Because the transfer precedes the callback, the revert rolls it back: this pool's income is booked, accumulates in the ProtocolShareReserve and cannot be paid out. No funds are lost, only undistributable. Shortfall keeps an independent single-registry binding as well, so bad-debt auctions still cannot start for this pool either.

Both contracts expose only a single-address setPoolRegistry, so repointing either at SpokePoolRegistry would break the existing isolated pools in exactly the way #168 exists to avoid. They need the same additional-registry mechanism, or a parallel RiskFundConverter and Shortfall bound to this pool's markets through VToken's per-market setters. Verify all three contracts resolve this pool before listing any market - #168's own fork suite covers thirteen cases on the booking side and calls releaseFunds in none of them.

Note: RiskFundConverter and Shortfall still bind a single poolRegistry and were not updated. Neither is on the live path: VIP-671 moves the risk-fund share from RiskFundConverter to RiskFundBuyback on bsctestnet (bscmainnet already did so in VIP-618), and Shortfall auctions are paused on bscmainnet by VIP-618.

[I01] Markets can be listed before the pool can price them: the pricing dependency this fork adds is unvalidated at both levels	
Contract	SpokeComptroller.sol
Severity	Informational
Description	supportMarket is the only path by which a market becomes reachable by users. It validates the caller, duplicate listing and vToken identity, but not whether the pool can price the market:

```javascript
// SpokeComptroller – supportMarket
function supportMarket(VToken vToken) external {
    _checkSenderIs(poolRegistry);
    ...
    newMarket.isListed = true;
}
```

The dependency this fork adds, deviationBoundedOracle, is unvalidated at two levels.

Pool-wide: the reference is dereferenced without a zero check. initialize does not set the field, and while PoolRegistry.addPool applies ensureNonzeroAddress to oracle, there is no equivalent guard here:

```javascript
// SpokeComptroller – _updateProtectionStates
IDeviationBoundedOracle boundedOracle = deviationBoundedOracle;
for (uint256 i; i < vTokensCount; ++i) {
    boundedOracle.updateProtectionState(address(vTokens[i]));
}
```

This is reached from preBorrowHook and from _checkRedeemAllowed (shared by preRedeemHook, preTransferHook and exitMarket), while preMintHook reads no oracle. Reproduced with the repository's own fixtures using setBoundedOracle: false: **deposit and enterMarkets pass, redeem and exitMarket revert** — and the revert comes from the extcodesize check, so it carries no error data.

Per asset: bounded pricing degrades silently to spot when not enabled.

```javascript
// DeviationBoundedOracle – _computeBoundedPrices
if (!state.isBoundedPricingEnabled) return (spot, spot);
```

The state cannot be inferred from the prices ("not enabled" and "enabled with a window equal to spot" return the same values); the only way to observe it is isBoundedPricingEnabled(underlying), which PoolLens does not expose. Uninitialised assets exist on chain today — the team's fork suite asserts isBoundedPricingEnabled(USDT) is false. That is correct under the intended configuration, since USDT is the liquidity side; the risk is a future listing that admits an unconfigured asset as collateral.

The contract already contains the exact precedent, simply not extended to the new dependency:

```
JavaScript
  // SpokeComptroller — setCollateralFactor
  if (newCollateralFactorMantissa != 0 &&
oracle.getUnderlyingPrice(address(vToken)) == 0) {
      revert PriceError(address(vToken));
  }
```

Impact:
Pool-wide: a market listed before setDeviationBoundedOracle lands accepts deposits that cannot be withdrawn until an owner transaction arrives, and the failure is a bare revert the frontend cannot explain. The same applies on upgrade — between upgradeTo and the setter, redeem and borrow break for every market member.

Per asset: if a collateral market's asset is not enabled, this fork's deviation-bounded protection does not apply to that market at all, borrowing capacity is computed at spot, and nothing in the contracts, events or lens output says so.

Recommendation

Pool-wide, add the readiness check to supportMarket and replace the implicit revert in _updateProtectionStates with a named error:

```
JavaScript
  if (address(deviationBoundedOracle) == address(0)) revert
BoundedOracleNotSet();
```

Per asset, extend the existing "nonzero collateral factor requires a usable price" check in setCollateralFactor to cover the second dependency:

```
JavaScript
  if (newCollateralFactorMantissa != 0 &&

!deviationBoundedOracle.isBoundedPricingEnabled(underlyingOf(vToken))) {
      revert BoundedPricingNotEnabled(address(vToken));
  }
```

Note that isBoundedPricingEnabled takes the underlying rather than the vToken, and the oracle resolves native and VAI markets specially, so having the oracle expose a view keyed on the vToken is the more robust approach. Adding the flag to SpokePoolLens's per-market output would also make the state observable to operators.

Status

Acknowledged. In efd1390, SpokePoolLens now reports boundedPricingEnabled per market, which makes the per-asset state observable. The recommended guards in SpokeComptroller were not added because of the EIP-170 size limit: the contract is 24,551 bytes, leaving 25 bytes against the 41 and 271 the guards require. The team instead relies on the listing order in VIP-671, where setPriceOracle and

setDeviationBoundedOracle are executed before addPool and addMarket, and bounded pricing is enabled for the collateral asset via setTokenConfig. The SpokePoolLens already deployed on bsctestnet must be redeployed for the new field to be available.

```javascript
function _boundedPricingEnabled(SpokeComptrollerViewInterface spokeView,
address underlying) private view returns (bool) {
   IDeviationBoundedOracle boundedOracle =
spokeView.deviationBoundedOracle();
   if (address(boundedOracle) == address(0)) return false;
   return boundedOracle.isBoundedPricingEnabled(underlying);
}
```

[I02] Empty-Registry Precondition Blocks Every Subsequent Deployment Run Once the Pool Is Registered	
Contract	024-deploy-spoke-pool-registry.ts
Severity	Informational
Description	The script deploys SpokePoolRegistry with skipIfAlreadyDeployed, then refuses to continue if the registry already holds any pool:

```javascript
// A fresh registry must be empty. A non-empty one means this name
resolved to a
// registry that is already in use, and pointing the spoke
implementation at it
// would defeat the separation this deployment exists for.
const registeredPools = await registry.getAllPools();
if (registeredPools.length !== 0) {
  throw new Error(
    `Refusing to transfer ownership: ${DEPLOYMENT_NAME} at
${registry.address} already ` +
      `holds ${registeredPools.length} pool(s), so it is not the empty
registry this ` +
      `deployment expects`,
  );
}
```

Neither 024 nor 025 sets func.id, so hardhat-deploy re-executes the script body on every `hardhat deploy` invocation; only the individual deploy() call is skipped. The check therefore runs again after the listing VIP has called addPool, at which point getAllPools().length is non-zero and the script throws before reaching the ownership

transfer. Every script ordered after it, including 025, is skipped. 025 additionally declares func.dependencies = [POOL_REGISTRY_NAME], so tag-scoped runs pull 024 in as well.
The condition the check defends against cannot arise: deploy("SpokePoolRegistry", { skipIfAlreadyDeployed: true }) either creates a new proxy or returns its own prior deployment record, and never resolves to the isolated-pools registry. That case already has a correct and specific guard in 025:

```javascript
const isolatedPoolRegistry = await
deployments.getOrNull("PoolRegistry");
if (isolatedPoolRegistry && sameAddress(poolRegistry.address,
isolatedPoolRegistry.address)) {
  throw new Error(
    `${POOL_REGISTRY_NAME} resolved to the isolated-pools PoolRegistry
at ` +
      `${isolatedPoolRegistry.address}. The spoke pool needs a registry
of its own; ` +
      `run 024-deploy-spoke-pool-registry.ts first.`,
  );
}
```

Impact
Once the pool is registered on chain, routine deployment runs on that network are blocked at 024 and no later script executes. No incorrect on-chain state results, but the deployment pipeline stops, and the fastest apparent remedy under time pressure is to comment the check out. Verified with a proof-of-concept run inside a single hardhat test session so chain state persisted across script runs: with the registry empty the re-run of 024 passed; after addPool raised getAllPools().length to one, the same re-run threw, with the error naming 024-deploy-spoke-pool-registry.ts. The empty-registry run is the control, isolating the registry contents as the cause.
Separately, `--tags HubSpoke` fails on a fresh network with "No deployment found for: AccessControlManager", because neither script declares the access control manager as a dependency, although 025's comment does address tag-scoped runs for the registry.

Recommendation

Remove the empty-registry check, since the case it guards is already covered by 025. If a precondition is still wanted, invert it into one that stays true after listing, for example asserting that the registry holds at most one pool and, if it holds one, that its comptroller is the Comptroller_HubSpoke deployment.

Independently, give 024 and 025 func.id values so they do not re-execute after a successful run, matching the convention already used by 012-transfer-pools-ownership.ts:

```javascript
func.id = "transfer_pools_ownership"; // id required to prevent re-execution
```

Finally, consider declaring AccessControlManager as a dependency of both scripts so that tag-scoped runs work on a fresh network.

Status	Fixed in commit 7e3c344. The empty-registry precondition in 024 was removed, so subsequent deployment runs no longer stop once the pool is registered, and both 024 and 025 now declare AccessControl as a dependency, so tag-scoped runs work on a fresh network. The func.id suggestion was not adopted: hardhat-deploy only uses func.id to skip a script that returns true, which neither script does, so it would have no effect; the team removed it in ca48d4f. Every step in both scripts checks the current state before acting, so re-runs remain safe.

[I03] The cross-repository interface guard sits behind the fork gate and never runs in CI, so the hub's hand-written copy can silently fall out of step	
Contract	SpokeComptrollerInterface.sol (SpokeComptrollerViewInterface); the guard lives in tests/hardhat/Fork/HubSpoke/interfaces.ts and the CI configuration in .github/workflows/ci.yaml
Severity	Informational
Description	The Liquidity Hub's AdapterSpokeV1 calls four getters on a spoke pool, declared by this repository's SpokeComptrollerViewInterface and implemented by SpokeComptroller. The hub lives in a separate repository, venus-liquidity-hub, and does not depend on this one, so it hand-writes its own interface of the same name declaring the same four functions. The two declarations must agree, and nothing enforces that at compile time. Solidity does not verify an interface declaration against the target contract — an external call compiles to "send this selector to this address and decode the return data per the declaration", and the compiler never checks that the function exists on the target. If this repository renames one of those getters or widens its return type, this repository compiles and its CI passes, the hub compiles against its stale declaration and its CI passes too, and the mismatch surfaces only as an on-chain revert when the hub makes the call. This PR wrote the correct guard for exactly that: interfaces.ts encodes the four members the hub depends on as data and compares this repository's interface declaration against its implementation, selector for selector and return type for return type. The test cannot read the hub repository's copy, but it does guarantee that this side keeps offering those four members with unchanged shapes — renaming, removing or widening any of them fails the test, which is the signal that the hub's side needs to be synchronised.

The issue is that the guard sits behind the fork gate, and none of the CI jobs sets that environment variable. test:fork is referenced nowhere in the workflow, so the mechanism only fires when someone runs the fork suite by hand.

```javascript
  // tests/hardhat/Fork/HubSpoke/interfaces.ts — HUB_REQUIRES and the
fork gate
  const HUB_REQUIRES = [
    { sig: "supplyCaps(address)", returns: "uint256" },
    { sig: "actionPaused(address,uint8)", returns: "bool" },
    { sig: "isSupplyAllowlistEnabled(address)", returns: "bool" },
    { sig: "isAllowedSupplier(address,address)", returns: "bool" },
  ];
  if (FORK && FORKED_NETWORK === "bscmainnet") {
    describe("HubSpoke: the hub-side interfaces against the real
contracts", () => {
      ...
```

Verified that FORK does not appear anywhere in .github/workflows/ci.yaml, and that none of its four jobs (lint, test, deploy, export-deployments) sets the variable.

Impact:
The guard exists but never runs automatically, so a breaking change to this interface is not caught by any automated step. The risk is not that a mismatch exists today — the declaration and the implementation currently agree — but that nothing automated will notice when they drift, and the mismatch will first appear as a failed on-chain call from the hub.

Recommendation

Configure an archive-node secret for CI and require interfaces.ts to run on any pull request that touches contracts/Spoke/SpokeComptrollerInterface.sol — a change on this side fails the test, which is precisely the signal that the hub's copy needs updating.

The remaining integration assertions can run on a schedule, or be triggered when contracts/Spoke/, contracts/test/HubSpoke/, or deploy/02* are touched.

Status

Fixed in commit 9916125. The ABI-level assertions (HUB_REQUIRES and the interface conformance checks) were moved out of the fork gate into tests/hardhat/Spoke/interfaces.ts, which runs under the ordinary CI test job on every pull request. Only the assertions that need a live pool remain in the fork suite.

```javascript
// tests/hardhat/Spoke/interfaces.ts — runs under the ordinary test job
const HUB_REQUIRES = [
  { sig: "supplyCaps(address)", returns: "uint256" },
  { sig: "actionPaused(address,uint8)", returns: "bool" },
  { sig: "isSupplyAllowlistEnabled(address)", returns: "bool" },
  { sig: "isAllowedSupplier(address,address)", returns: "bool" },
];
```

[I04] reduceReservesBlockDelta is declared for the spoke markets but no deployment step applies it	
Contract	deploy/028-deploy-spoke-vtokens.ts, helpers/spokeDeploymentConfig.ts
Severity	Informational

Description	reduceReservesBlockDelta throttles how often a market sweeps its accrued protocol share to the ProtocolShareReserve. The isolated pools set it to 100 blocks. For the spoke markets it is never set and stays at the zero default: VToken._initialize does not assign it, 028 does not pass it among the eleven arguments it gives initialize, and the only script that issues the call iterates a configuration file the spoke pool is deliberately kept out of.
	
	JavaScript
	`// deploy/013-vip-based-config.ts - the isolated markets get it, ahead of listing`
	`return [`
	`  ...(await transferInitialLiquidity(vTokenConfig, deploymentConfig)),`
	`  ...(await approvePoolRegistry(poolRegistry, vTokenConfig, deploymentConfig)),`
	`  await setReduceReservesBlockDelta(vToken.address, vTokenConfig),`
	`  await addMarket(poolRegistry, vToken.address, vTokenConfig),`
	`];`
	
	The value is declared in helpers/spokeDeploymentConfig.ts but nothing reads it. That file intentionally carries both the parameters 028 consumes and the parameters the listing VIP needs, so a field being unread is normal in itself. What distinguishes this one is that every other VIP parameter is a mandatory argument of a call the VIP must make - closeFactor, liquidationIncentive and minLiquidatableCollateral belong to addPool, and collateralFactor, liquidationThreshold, initialSupply, vTokenReceiver, supplyCap and borrowCap are the six non-address members of AddMarketInput - so none of them can be silently skipped. reduceReservesBlockDelta is the only value that needs a separate, standalone call.
	
	JavaScript
	`// PoolRegistry - every other market parameter is a required argument`
	`struct AddMarketInput {`
	`    VToken vToken;`
	`    uint256 collateralFactor;`
	`    uint256 liquidationThreshold;`
	`    uint256 initialSupply;`
	`    address vTokenReceiver;`
	`    uint256 supplyCap;`
	`    uint256 borrowCap;`

```
}
```

At zero the branch in accrueInterest is always true, so the sweep runs on every accrual rather than once per window:

```javascript
// VToken - accrueInterest
if (currentSlotNumber - reduceReservesBlockNumber >=
reduceReservesBlockDelta) {
    reduceReservesBlockNumber = currentSlotNumber;
    _reduceReservesFresh(cashPrior < totalReservesNew ? cashPrior :
totalReservesNew);
}
```

Impact
Every state-changing operation on a spoke market carries an extra ERC-20 transfer and an external call into the ProtocolShareReserve. Measured against the same market with the throttle set to 100, a mint costs 232,545 gas instead of 174,526, about 33% more; the figure is a lower bound, since the measurement used a mock reserve contract. The sweep does not affect any computed value - cash and reserves fall by the same amount, so the exchange rate, the utilisation the rate model reads and the redeem and borrow liquidity gates are all unchanged - and the ProtocolShareReserve receives many small transfers instead of one per window. Note also that the team's own handover note at the end of 028, listing what the VIP still owes these markets, does not mention this value.

Recommendation

Give the value a consumer so that a wrong or missing entry fails visibly, as it does for every other market parameter. The simplest form is to apply it in 028 straight after the market is deployed, since that script already reads the same configuration file:

```javascript
// deploy/028-deploy-spoke-vtokens.ts, after the market is verified
await
vToken.setReduceReservesBlockDelta(vTokenConfig.reduceReservesBlockDelta
);
```

Alternatively, add the call to the listing VIP ahead of addMarket, matching the order the isolated pools use. Either way the value should reach the chain through the same path that carries it in the configuration, rather than depending on the VIP author remembering a step that no other parameter requires.

Status

Acknowledged. setReduceReservesBlockDelta is governance-gated, so a deploy script cannot call it. The team sets it to 100 in VIP-671 ahead of addMarket, which is the same order the isolated pools use, and documented the requirement in 028 in commit 5c74999.